Assignment 1: Parallel Numerical Integration

CS5741 Concurrency and Parallelism in Software Development, AY26-27 Semester 1

Chirag Aggarwal · ID 26253925

26253925@studentmail.ul.ie

Lecturer: Faeq Alrimawi

Contents

1 Foster's Methodology

2 Parallel Implementation

3 Machine Specifications

4 Performance Analysis

5 Conclusion

References

AI Usage Statement

Notation

n = number of intervals
h = (b − a) / n
p = number of threads
s = serial fraction
Amdahl's Law: S(p) = 1 / (s + (1 − s) / p)
Maximum speedup = 1 / s

1. Foster's Methodology

1.1 Partitioning

Partitioning is the first step of Foster's methodology: divide and conquer.

The function to integrate is $f(x) = e^{-x} \cdot \sin(x)$. Since the computation is already confined, the problem is split by **domain (data) decomposition**.

The domain is divided into n + 1 sample points, $x_i = a + i \cdot h$, where:

- x_i = x-coordinate of the i-th point
- a = start of the interval
- b = end of the interval
- n = number of trapezoids
- h = width of each strip
- i = index

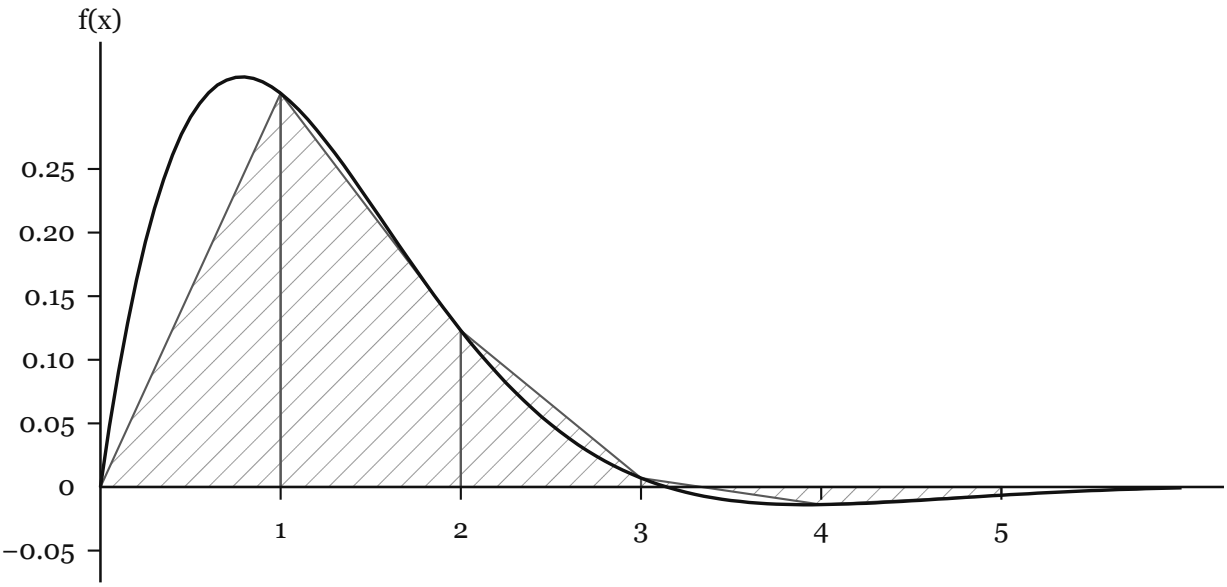


Figure 1: Diagram: sketch of f(x) with the area under the curve split into trapezoid strips

- Each task is independent: it is just a computation.
- The tasks are uniform: each one evaluates e^{-x} and sin, then multiplies them.
- Special case: the endpoints i = 0 and i = n have weight ½.

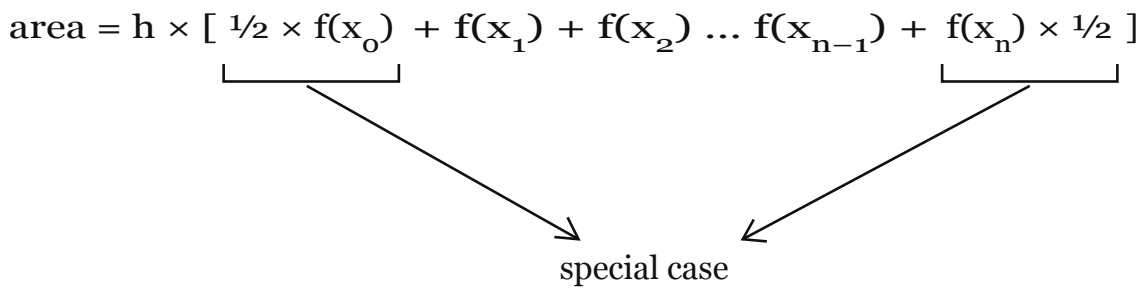


Figure 2: Diagram: trapezoid area formula with the first and last terms marked as the special case

Why do the endpoints have weight ½?

The area of one trapezoid strip is h × (left height + right height) / 2, so:

strip 1 = $h/2 \times (f(x_0) + f(x_1))$

strip 2 = $h/2 \times (f(x_1) + f(x_2))$

⋮

Every interior point is shared by two neighbouring strips, so its two halves combine into a full weight. Only the first and last points keep a weight of ½.

1.2 Communication

The communication stage defines the data flow between the primitive tasks.

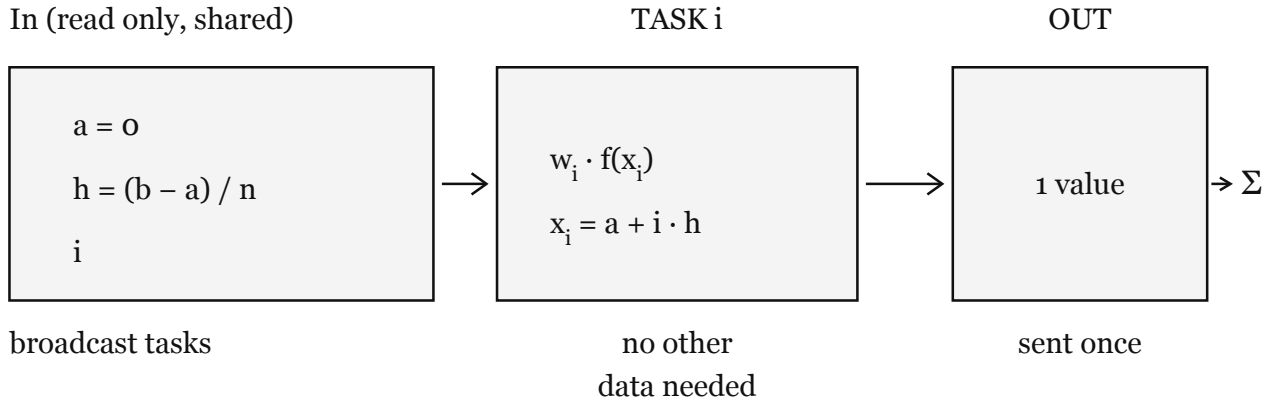


Figure 3: Diagram: data flow for one task, from shared read-only inputs to a single output value

The weights are $w_i = ½$ if i = 0 or i = n, and $w_i = 1$ otherwise.

There is no communication between tasks. Each task needs only the shared, read-only inputs, and sends its single value once. The only communication is a global reduction:

$$\text{area} = h \times \Sigma\, w_i \cdot f(x_i)$$

Following Foster, a **tree reduction** is used for this sum:

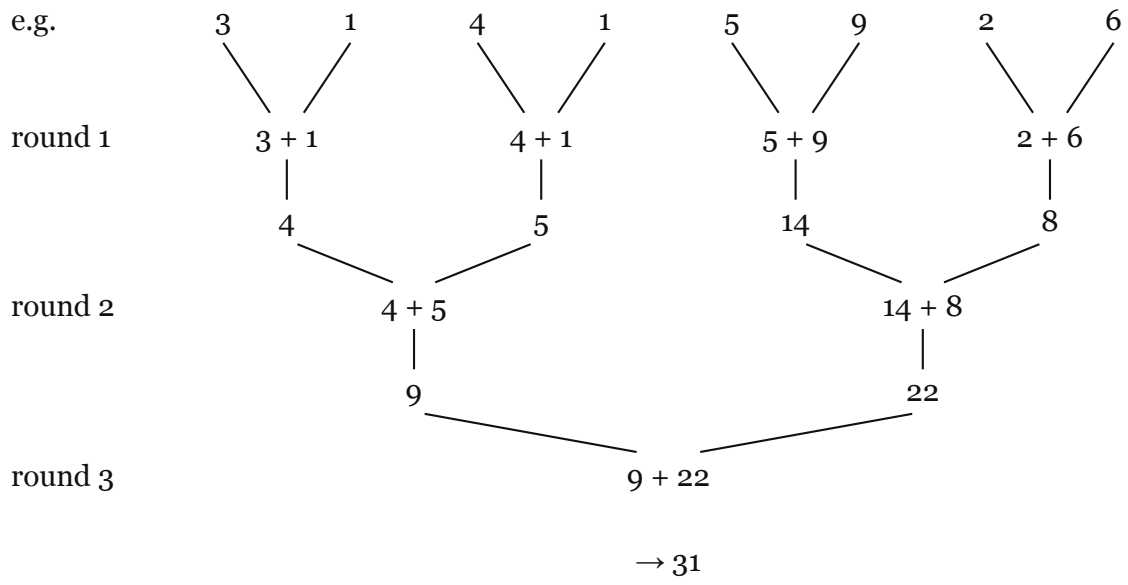


Figure 4: Diagram: tree reduction of eight example values over three rounds

With eight values this takes only 3 steps (steps = $\log_2$ n).

1.3 Agglomeration

Take a problem size of 500m (500 million) as an example. One thread per task would be too fine-grained, since the actual work would be 500,000,000 × (exp × sin + x).

Block grouping

Block size = (n − 1) / p, where n = number of intervals and p = number of threads. The n − 1 interior points are shared out; the two endpoints are added once by the main thread.

Each thread gets one contiguous chunk of the interior points (tasks) x_1, x_2 … x_{n-1}.

This reduces the number of values to combine:

- before: n + 1 values to reduce
- now: p values to reduce (one partial sum per thread)

Block vs cyclic distribution

Block	[0 0 0 0] [1 1 1 1] [2 2 2 2]	✓ chosen
Cyclic	[0 1 2 0 1 2 0 1 2 0 1 2]	✗ rejected

A cyclic distribution is only useful if some parts of the task are more expensive than others.

1.4 Mapping

In Java, mapping happens in two steps: we decide how blocks are assigned to threads, and the JVM and operating system decide how threads are placed on CPU cores.

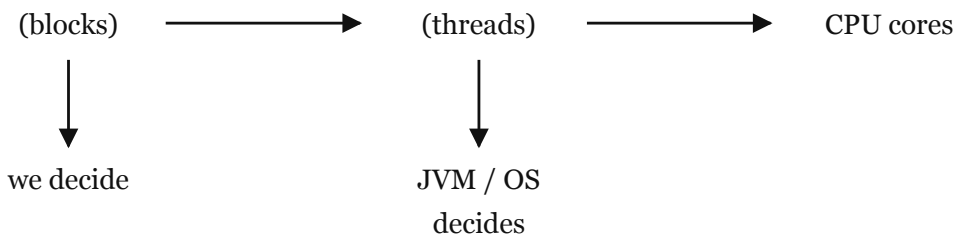


Figure 5: Diagram: blocks are mapped to threads, and threads to CPU cores

Mapping can be static or dynamic. Since the work here is equal across all tasks, static mapping is chosen.

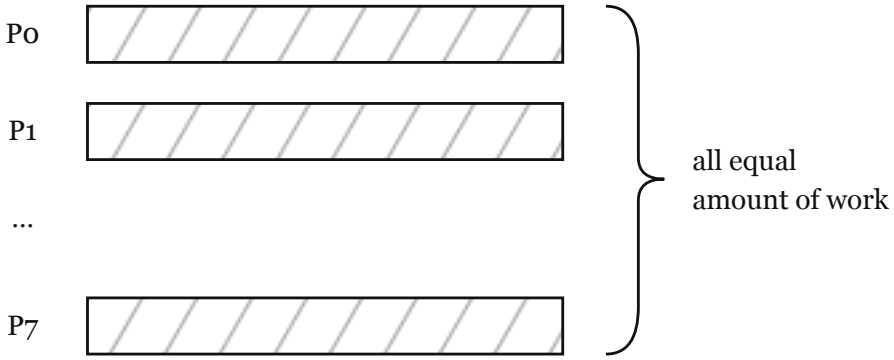


Figure 6: Diagram: threads P0 to P7 each given an equal block of work

1.5 Summary

1. **Partitioning:** domain (data) decomposition
   - n + 1 tasks (n = 10m, 100m, 500m)
   - the same operation for every task
2. **Communication:** none during the computation
   - one global sum reduction at the end
   - a tree reduction takes $\lceil \log_2 n \rceil$ rounds, e.g. 24 for 10m
3. **Agglomeration:** contiguous blocks
   - p blocks of about (n – 1) / p points each (p = number of threads)
4. **Mapping:** static mapping
   - the tasks are uniform

2. Parallel Implementation

Each part of the Foster design maps directly to a part of the Java code:

Foster design	Java code
Independent evaluations	`f(a + i * h)`
Block grouping	start / end bounds
One block per thread	`new Thread(...)`
Independent computation	`localSum`
Collect each block result	`partialSums[worker]`
Combine / reduce	`join()` → loop adding `partialSums`

Thread management

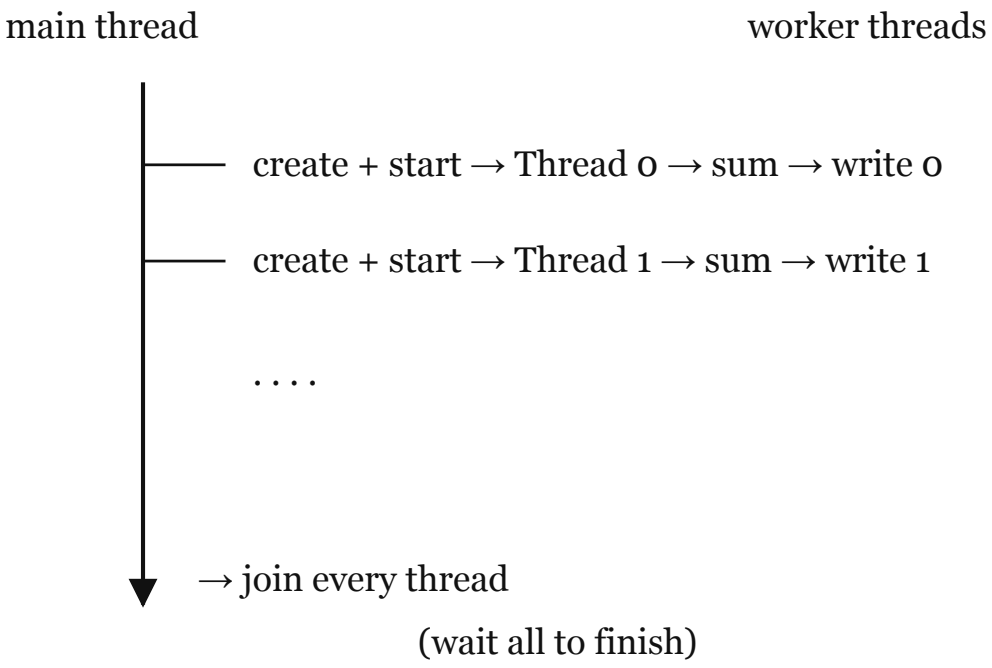


Figure 7: Diagram: main thread creating and starting worker threads, then joining them

After all threads have joined, the main thread combines the partial sums, multiplies by *h* and returns the result.

Synchronisation

- None is needed, as all tasks are independent.
- No locks and no atomics are used.
- Only `join()` is needed, to wait for the threads.

Correctness

The range calculation works on $m = n - 1$ interior points. Thread t gets:

start = 1 + floor($t \times m / p$)
end = 1 + floor($(t + 1) \times m / p$)

If $n \leq 0$ or $p \leq 0$, an `IllegalArgumentException` is thrown.

Verification

n	***p***	**Result vs serial**
1	4	✓
11	3	✓
3	8	✓
1,000,000	4	✗ a very small difference, which is expected

3. Machine Specifications

CPU	Apple M2 Pro
Cores	12 (8 performance + 4 efficiency); each core runs one thread at a time
L2 cache	16 MB (performance cluster), 4 MB (efficiency cluster)
Memory	32 GB
OS	macOS 26.6.2
Java	OpenJDK 27

The M2 Pro has 8 performance cores and 4 efficiency cores, with no SMT (simultaneous multithreading). So there are 12 cores, and the 16-thread runs use 4 more threads than cores.

Speedup for n = 500m:

Threads	2	4	8	12	16
Speedup	1.91×	3.68×	6.20×	7.76×	7.83×

At 2 and 4 threads the speedup is close to ideal. From 8 threads on it is much smaller in comparison, and going from 12 to 16 threads adds almost nothing because there are only 12 cores.

4. Performance Analysis

4.1 Method

Each configuration was run 7 times after 3 warm-up rounds, and the median is reported. Runs were made on AC power.

4.2 Execution time (median, ms)

n	serial	p = 2	p = 4	p = 8	p = 12	p = 16
10M	106.4	56.4	28.4	20.2	17.5	17.6
100M	1,067.6	561.5	286.2	181.2	144.4	144.9
500M	5,362.3	2,804.1	1,458.7	865.2	691.1	684.8

4.3 Actual speedup, $S = T_{serial} / T_{parallel}$

n	p = 2	p = 4	p = 8	p = 12	p = 16
10M	1.89	3.74	5.26	6.07	6.03
100M	1.90	3.73	5.89	7.39	7.37
500M	1.91	3.68	6.20	7.76	7.83

4.4 Efficiency, S / p

n	p = 2	p = 4	p = 8	p = 12	p = 16
10M	94%	94%	66%	51%	38%
100M	95%	93%	74%	62%	46%
500M	96%	92%	77%	65%	49%

4.5 Fixed thread overhead (n = 1, every range empty; median of 200 runs, ms)

p = 2	p = 4	p = 8	p = 12	p = 16
0.211	0.311	0.567	0.868	1.151

4.6 Amdahl's Law

Amdahl's law gives the speedup as:

$$S(p) = 1 / (s + (1 - s) / p)$$

where s = serial (non-parallelisable) fraction, (1 − s) = parallelisable fraction, p = number of threads and S(p) = speedup. The maximum possible speedup is 1 / s.

For this program, the stages are:

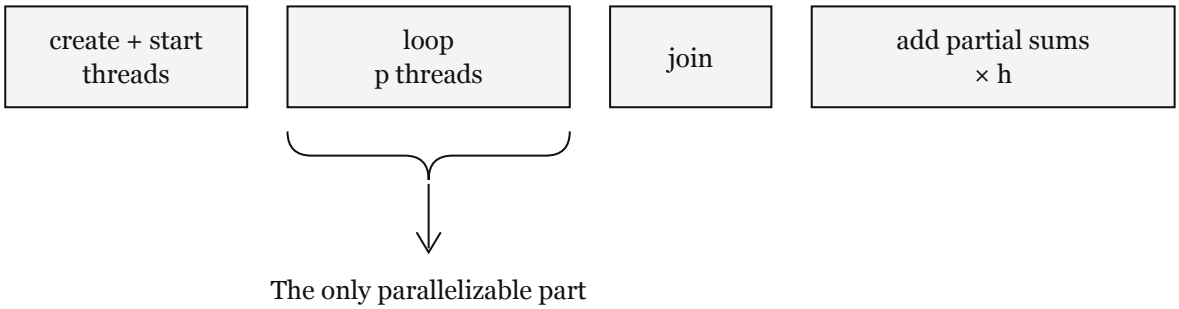


Figure 8: Diagram: program stages in order, with the loop over p threads marked as the only parallelizable part

The serial fraction is estimated as $s = T_{overhead}(p = 16) / T_{serial}(n)$:

n	Serial time	s
10m	106 ms	0.0108 (1.1%)
100m	1,068 ms	0.0011 (0.11%)
500m	5,362 ms	0.0002 (0.02%)

Since the overhead is fixed at about 1 ms, a bigger problem gives a smaller s.

Predicted speedups

n	p = 2	p = 4	p = 8	p = 12	p = 16	max (1 / s)
10m	1.98	3.87	7.44	10.73	13.77	93
100m	2.00	3.99	7.94	11.86	15.74	909
500m	2.00	4.00	7.99	11.97	15.95	5,000

The predictions barely change with problem size. Because s is very small, Amdahl's law allows a very high maximum speedup (93× at 10m, 909× at 100m and 5,000× at 500m), but the hardware is limited. Amdahl's assumptions do not hold on this machine:

Assumption	Reality
All cores are equally fast	8 performance and 4 efficiency cores
Number of cores = number of threads	We ran up to 16 threads on 12 cores
Cores perform at the same speed	Clock speeds do reduce (overhead)

4.7 Graphs

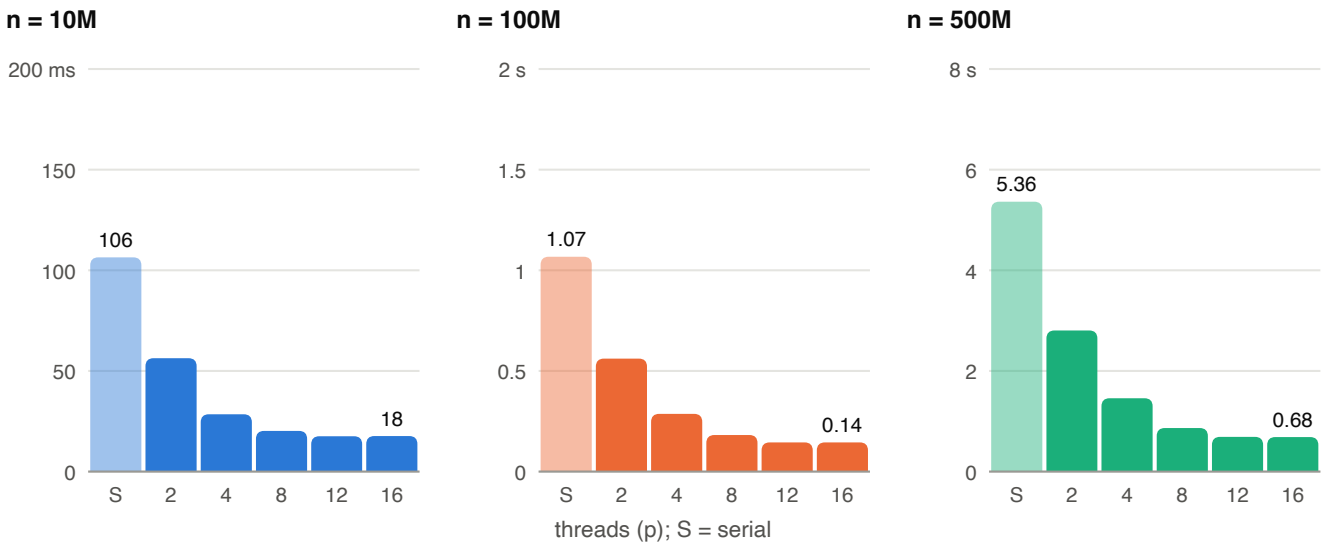


Figure 9: Median execution time for each problem size (S = serial). Each panel has its own scale.

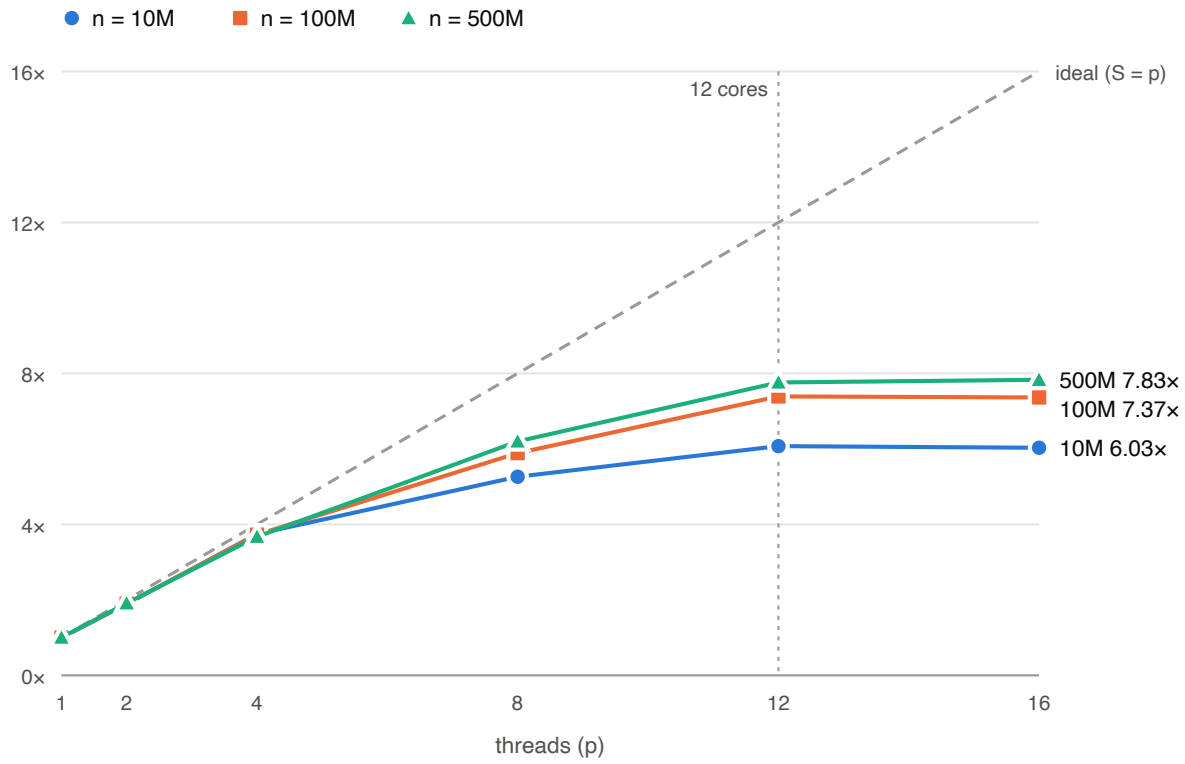


Figure 10: Actual speedup against thread count, with ideal linear speedup and the 12-core limit.

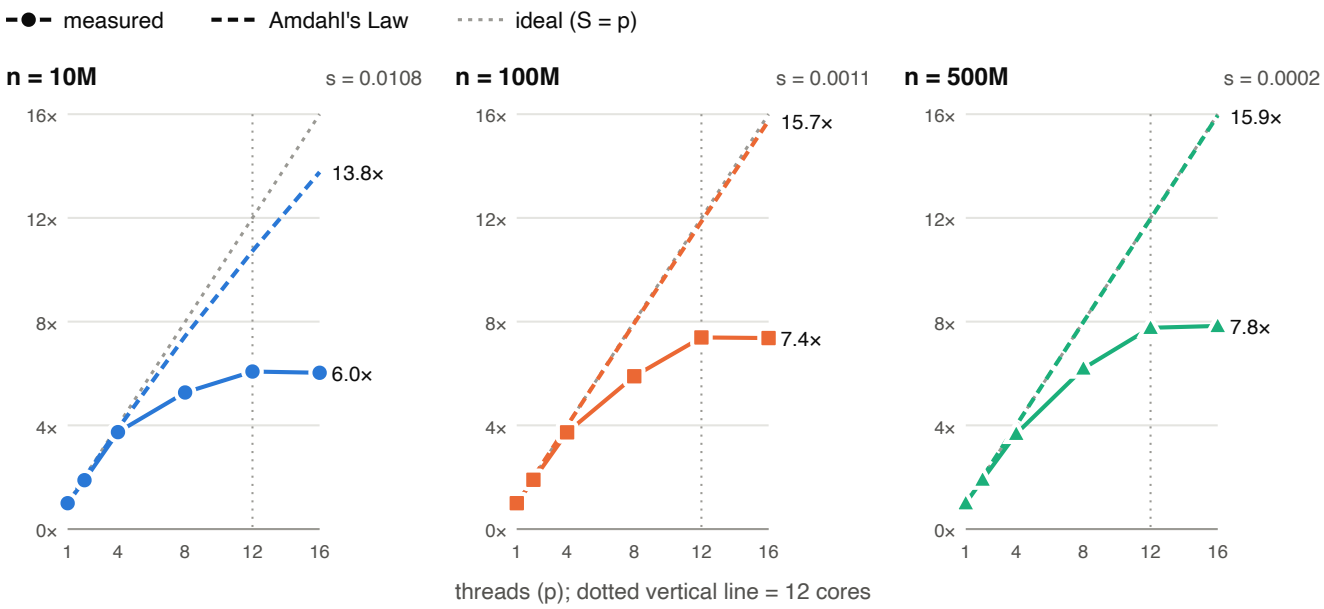


Figure 11: Measured speedup against Amdahl's Law for each problem size, using $s = T_{overhead}(p = 16) / T_{serial}(n)$. At 100M and 500M the Amdahl line lies almost on top of the ideal line.

4.8 Discussion

Overall result

The best speedup was 7.8× (n = 500m, p = 12–16) on my 12-core M2 Pro. Scaling was linear up to p = 4, and flattened after that. Efficiency at each thread count:

Threads	Efficiency
p = 2–4	92–96%
p = 8	66–77%
p = 12	51–65%
p = 16	38–49%

Effect of problem size

The program scales better with bigger problems. The overhead is fixed at about 1 ms, so there are no scaling issues from it.

Amdahl vs reality

Amdahl's law predicts a speedup of 13.8–15.9× at p = 16, but we measured 6.0–7.8×. This gap is mostly down to the hardware.

Evaluation of design choices

- ✓ No locks or atomics, only a single `join()` per thread.
- ✓ Block grouping.
- ? Static mapping: maybe not the best choice, as the cores were uneven.

5. Conclusion

I parallelised trapezoidal-rule integration of $e^{-x} \cdot \sin(x)$ on [0, 10] using Foster's methodology, implemented it in Java, and checked the results against Amdahl's law.

- **Design:** domain (data) decomposition, block grouping and static mapping.
- **Correctness:** very good, with an error of only about $1e^{-13}$ for very large n.
- **Main result:** around 7.8× speedup at n = 500m, linear speedup up to p = 4, and a very small serial part (overhead).
- **Amdahl's law** showed that the limitation was the hardware.

Possible improvement

Static mapping could be replaced, since not all the cores were equal.

References

1. Alrimawi, F. (2026a) *CS5741 Assignment 1: Parallel Numerical Integration* [assignment brief and supplied `SerialIntegration.java`]. University of Limerick, Brightspace, 8 October.
   Task specification, marking criteria, the trapezoidal rule and the serial baseline code.

2. Alrimawi, F. (2026b) *Lecture 2: Identifying Parallelisable Problems* [lecture slides], CS5741. University of Limerick.
   Decomposability, independence and granularity; data vs task parallelism; Amdahl's Law (slides 42–47); strong vs weak scaling (slide 48).

3. Alrimawi, F. (2026c) *Lecture 3: Parallel Algorithms* [lecture slides], CS5741. University of Limerick.
   ForkJoin and estimating the serial fraction (slides 9–10); task dependencies (RAW/WAR/WAW); static vs dynamic load balancing; Foster's methodology and its four checklists (slides 24–39); parallel patterns (slide 40).

4. Alrimawi, F. (2026d) *Lab 1: Sequential vs Concurrent vs Parallel* [lab handout and code], CS5741. University of Limerick.
   Shown in practice: a shared, synchronised accumulator serialises the threads.

5. Amdahl, G.M. (1967) 'Validity of the single processor approach to achieving large scale computing capabilities', in *Proceedings of the AFIPS Spring Joint Computer Conference*, Atlantic City, 18–20 April, pp. 483–485. doi:10.1145/1465482.1465560.
   Original statement of the speedup limit used in Section 4.

6. Apple Inc. (2023) 'Apple unveils M2 Pro and M2 Max: next-generation chips for next-level workflows', *Apple Newsroom*, 17 January. Available at: https://www.apple.com/newsroom/2023/01/apple-unveils-m2-pro-and-m2-max-next-generation-chips-for-next-level-workflows/ (accessed 14 October 2026).
   Core configuration of the test machine (performance and efficiency cores).

7. Foster, I. (1995) *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*. Reading, MA: Addison-Wesley.
   Source of the four-stage design methodology (partitioning, communication, agglomeration, mapping) applied in Section 1.

8. Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A., Smith, D. and Bierman, G. (2023) *The Java Language Specification, Java SE 21 Edition*, §17.4.4–17.4.5. Oracle. Available at: https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html (accessed 14 October 2026).
   Memory-model guarantee that all actions in a thread happen-before another thread returns from `join()` on it, which is why the partial sums are visible without locks.

9. Grama, A., Gupta, A., Karypis, G. and Kumar, V. (2003) *Introduction to Parallel Computing*. 2nd edn. Harlow: Addison-Wesley.
   Decomposition techniques (recursive, data, exploratory, speculative), used when considering alternative partitions.

10. Oracle (2023) *Class Thread*, Java SE 21 API documentation. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html (accessed 14 October 2026).
    `Thread` creation, `start()` and `join()` as used in the implementation.

11. Anthropic (2026) *Claude* [AI assistant]. Available at: https://claude.ai (accessed 10–19 October 2026).
    Used within the limits of section 4 of the brief; see the AI Usage Statement for exactly how.

AI Usage Statement

I used Claude (Anthropic, 2026), an AI assistant, during this assignment within the limits set in section 4 of the brief. This is how it was used.

Area	What the AI did	What I did
Understanding the problem	Explained the trapezoidal rule, the integrand $e^{-x}\cdot\sin(x)$, the endpoint weights and terms such as global reduction, block grouping and SMT. Built two interactive explainer pages (the curve with its trapezoids, and agglomeration).	Studied the explanations and the lecture slides.
Foster's methodology (Section 1)	Gave outlines for all four stages, cross-checked against the Lecture 3 checklists. Wrote example drafts for the Partitioning and Communication stages, and reviewed my Partitioning notes.	Wrote all of Section 1 by hand in my own words, including the sketches.
Parallel implementation (Section 2, Java file)	Supplied a skeleton: the header comment, the supplied serial method, the timing and printing in `main`, and numbered TODO comments. Reviewed my finished code and tested it against the serial version on 64 (n, p) combinations.	Wrote the `parallel()` method: range bounds, thread creation, local sums, `join()` and the reduction. Wrote Section 2 by hand in my own words.
Machine specifications (Section 3)	Read the hardware details from the machine and explained what they mean.	Wrote the notes in Section 3 by hand in my own words.
Performance measurement (Section 4)	Wrote and ran the benchmark harness (warm-up, median of 7 runs, the thread-overhead measurement at n = 1). Generated the result tables and graphs in Section 4 from the measured data.	Chose the experiments, including the extra p = 12 comparison.
Amdahl's Law (Section 4.6)	Explained the law, and calculated the serial fraction s, the predicted speedups and the maximum speedup from the measured data.	Wrote Section 4.6 by hand in my own words, including the comparison of Amdahl's assumptions with the real hardware.
Discussion and conclusion (Sections 4.8 and 5)	Gave outlines listing the points to cover and the supporting numbers from the results.	Wrote Sections 4.8 and 5 by hand in my own words.
Report production	Downloaded the brief and lecture slides from Brightspace, typed up my handwritten pages and edited them lightly for grammar and formatting, pointed out slips (arithmetic errors, numbers that did not match the measured results, a mistake in the Amdahl table and inconsistent notation), which I asked it to correct, redrew my hand sketches as diagrams, assembled this PDF and formatted the reference list.	Decided the structure and content of the report, and checked the typed text against my handwritten originals.

I checked all AI output for accuracy before using it, and I can explain every part of the design, the code and the results.